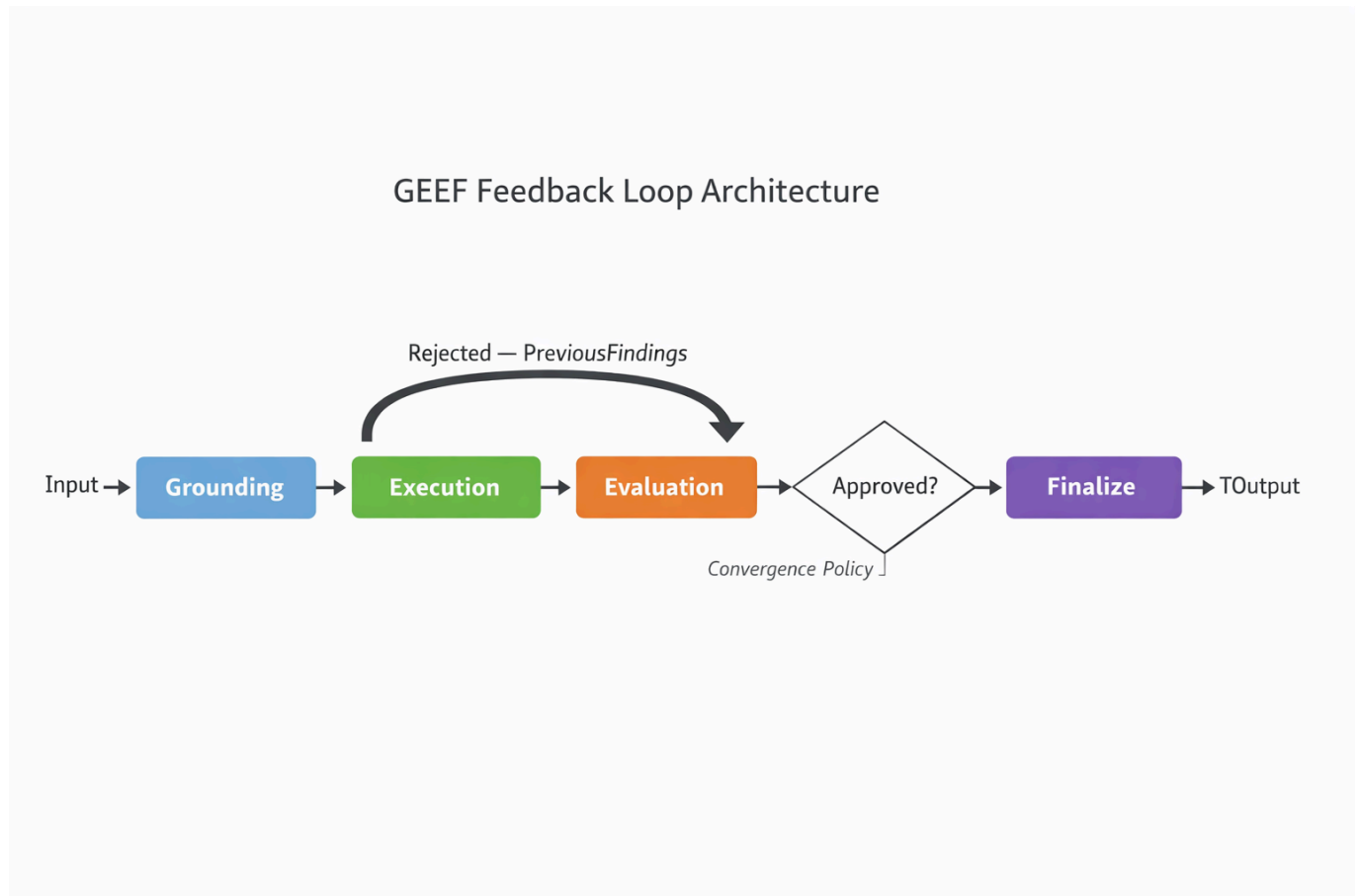


Konzepte und Architektur

Das GEEF-Pattern

GEEF steht für **Grounding** → **Execution** → **Evaluation** → **Finalize** und beschreibt einen typisierten, beobachtbaren, policy-gesteuerten Feedback-Loop für KI-Aufgaben.

Der Loop im Überblick



Grounding wird genau einmal ausgeführt. Der **Execution** ↔ **Evaluation**-Loop wiederholt sich, bis entweder alle Reviewer zustimmen oder die Konvergenz-Policy abbricht. **Finalize** wird nur bei Erfolg aufgerufen.

Die vier Phasen

1. Grounding

Die Grounding-Phase sammelt den gesamten Kontext, der für die Aufgabe benötigt wird:

- RAG-Abfragen (Retrieval-Augmented Generation)
- Dateisystem-Zugriffe

- API-Aufrufe
- Datenbank-Abfragen
- Konfigurationswerte

Eingabe: Ein String (der User-Prompt oder die Aufgabenbeschreibung). **Ausgabe:** Ein initialisierter `IRunContext` mit allen gesammelten Informationen.

```
public interface IGroundingStep
{
    Task<GroundingResult> RunAsync(string input, CancellationToken ct = default);
}
```

2. Execution

Die Execution-Phase erzeugt oder modifiziert Artefakte. Ab Iteration 2 erhält sie die `PreviousFindings` aus der vorherigen Evaluation — so kann sie gezielt nachbessern.

Eingabe: Der aktuelle `IRunContext`. **Ausgabe:** Ein neuer `IRunContext` (Immutabilität!) mit den erzeugten/modifizierten Artefakten.

```
public interface IExecutionStep
{
    Task<ExecutionResult> RunAsync(IRunContext context, CancellationToken ct = default);
}
```

3. Evaluation

Ein oder mehrere **Reviewer** prüfen die Artefakte unabhängig voneinander. Jeder Reviewer gibt eine `ReviewDecision` und optional `Finding`-Objekte zurück.

```
public interface IReviewer
{
    string Name { get; }
    int Priority { get => 100; }
    Task<ReviewResult> ReviewAsync(IRunContext context, CancellationToken ct = default);
}
```

Die Art, wie Reviewer ausgeführt werden, bestimmt die **Evaluation Strategy** (sequentiell, parallel, fail-fast, etc.).

4. Finalize

Wird nur aufgerufen, wenn alle Reviewer zugestimmt haben. Produziert das typisierte Endergebnis.

```
public interface IFinalizer<TOutput>
{
    Task<FinalizeResult<TOutput>> FinalizeAsync(IRunContext context, CancellationTok
    en ct = default);
}
```

Das Context-System

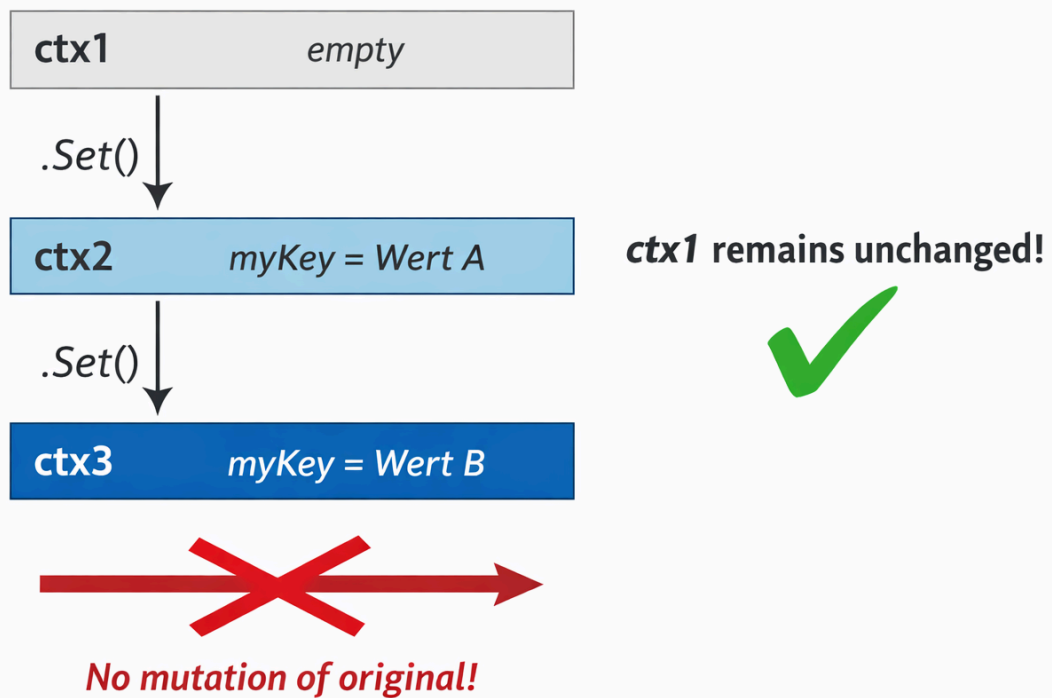
Immutabilität als Kernprinzip

`IRunContext` ist **unveränderlich**. Jeder `Set()`-Aufruf erzeugt einen neuen Snapshot:

```
var ctx1 = new RunContext();
var ctx2 = ctx1.Set(myKey, "Wert A");    // ctx1 ist unverändert!
var ctx3 = ctx2.Set(myKey, "Wert B");    // ctx2 ist unverändert!

// ctx1: leer
// ctx2: myKey = "Wert A"
// ctx3: myKey = "Wert B"
```

Immutable Context Snapshots



Warum Immutabilität?

- **Thread-Sicherheit:** Mehrere Reviewer können gleichzeitig denselben Context lesen, ohne Locks.
- **Nachvollziehbarkeit:** Jede Phase arbeitet mit einem definierten Snapshot. Seiteneffekte sind ausgeschlossen.
- **Reproduzierbarkeit:** Der Context zu jedem Zeitpunkt ist deterministisch rekonstruierbar.

Typisierte Keys

Keys sind generisch typisiert — Tippfehler und Typverwechslungen werden zur Kompilierzeit erkannt:

```
var codeKey    = new ContextKey<string>("generated:code");
var scoreKey   = new ContextKey<double>("review:score");
var tokensKey  = new ContextKey<int>("usage:tokens");

// Typsicher: context.Set(scoreKey, "falsch") → Kompilierfehler!
context.Set(scoreKey, 0.95); // OK
```

Vordefinierte Keys (GeefKeys)

Key	Typ	Beschreibung
OriginalInput	string	Der ursprüngliche Eingabe-String
PreviousFindings	IReadOnlyList<Finding>	Findings der letzten Evaluationsrunde
CurrentIteration	int	Aktuelle Iterationsnummer (1-basiert)
RunStartedAt	DateTimeOffset	Startzeit des Pipeline-Laufs
RunId	string	Eindeutige Lauf-ID (12 Zeichen)
IterationHistory	IterationHistory	Gesamte Iterationshistorie

Konvergenz

Was ist Konvergenz?

Konvergenz bedeutet, dass der Loop zu einem akzeptablen Ergebnis kommt — oder kontrolliert abbricht, wenn das nicht gelingt.

Konvergenz-Entscheidungen

Entscheidung	Bedeutung
Approved	Alle Reviewer zufrieden. Loop endet, Finalize beginnt.
Continue	Fehler gefunden, aber Fortschritt erkennbar. Weiter iterieren.
StopMaxAttemptsReached	Maximale Iterationen/Zeit überschritten. Abbruch.
StopStagnant	Dieselben Findings über N Iterationen. Keine Verbesserung.
StopRegression	Bereits behobene Fehler sind zurückgekehrt.
AbortCriticalBlocker	Kritischer Fehler erkannt. Sofortiger Abbruch.
EscalateToHuman	Automatische Lösung nicht möglich. Mensch muss eingreifen.

Stagnations-Erkennung

GEEF erkennt Stagnation über **Finding-Fingerprints**. Jeder Finding hat einen eindeutigen Fingerprint (z.B. Hash aus Reviewer-Name + Kategorie + Nachricht). Wenn sich die Fingerprint-Menge über `StagnationThreshold` Iterationen nicht ändert, bricht der Loop ab.

Regressions-Erkennung

GEEF erkennt Regression, wenn ein Fingerprint in Iteration N vorhanden war, in Iteration N+1 verschwand und in Iteration N+2 wieder auftaucht. Das deutet auf instabile Fixes hin.

Findings und Severity

```
public sealed record Finding
{
    public string ReviewerName { get; init; }
    public string Fingerprint { get; init; }    // Für Stagnation/Regression
    public string Message { get; init; }
    public FindingSeverity Severity { get; init; }
    public string? Category { get; init; }      // z.B. "Security", "Style"
    public string? ArtifactReference { get; init; }
}
```

Severity	Blockiert Loop?	Beschreibung
Info	Nein	Informativ. Wird protokolliert, blockiert nicht.
Warning	Nein	Warnung. Nicht blockierend.
Error	Ja	Fehler. Loop iteriert erneut.
Critical	Sofortiger Abbruch	Sicherheits-/Compliance-Problem. Pipeline stoppt.

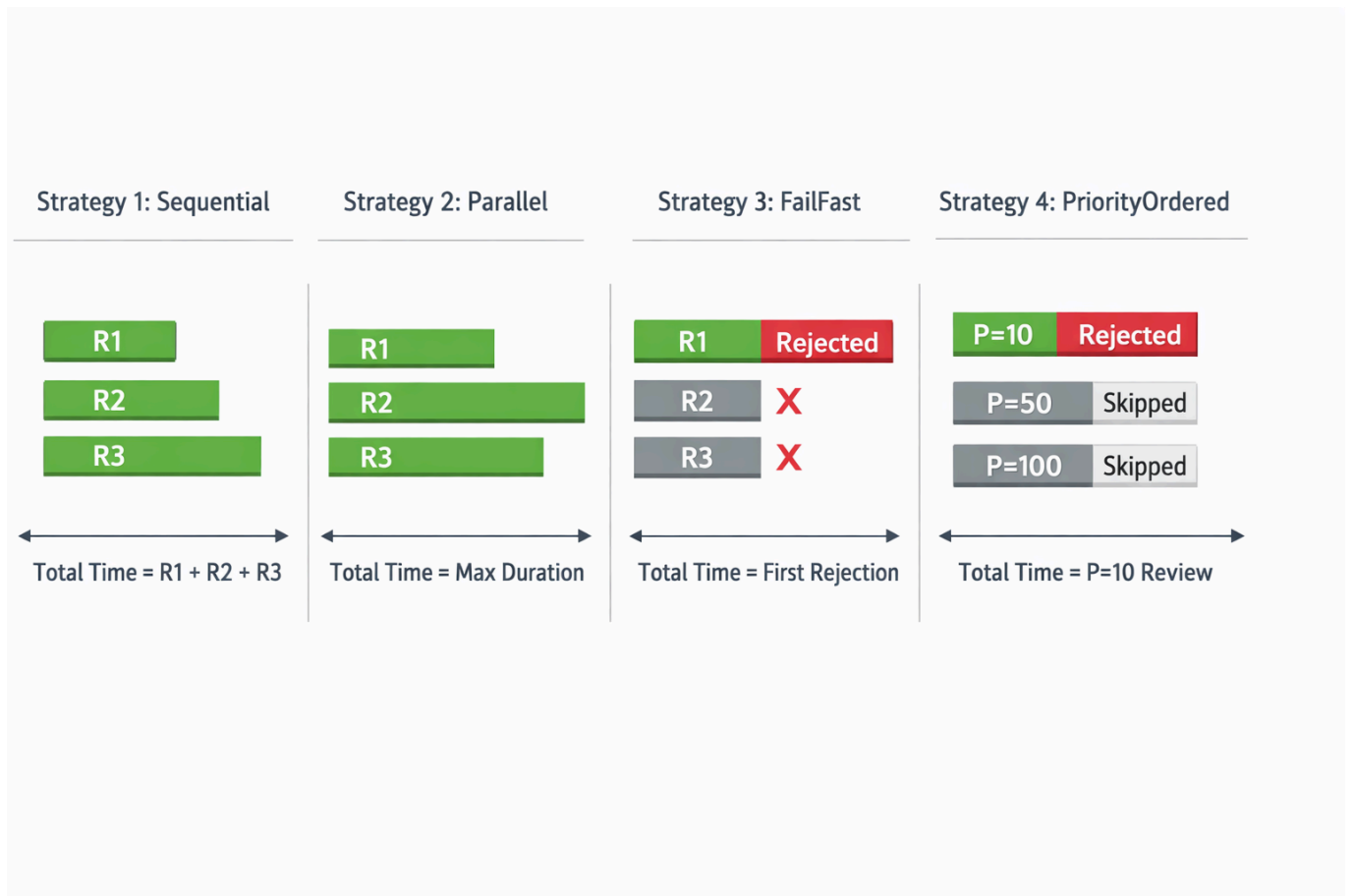
Evaluation Strategies

Die Evaluation Strategy bestimmt, **wie** Reviewer ausgeführt werden:

Strategie	Verhalten
SequentialEvaluationStrategy	Reviewer nacheinander. Einfach, deterministisch.
ParallelEvaluationStrategy	Alle Reviewer gleichzeitig via Task.WhenAll. Schnellste Gesamtausführung.
FailFastEvaluationStrategy	Parallel, aber sofortiger Abbruch beim ersten Rejected/Failed. Spart Kosten.
PriorityOrderedEvaluationStrategy	Sequentiell nach Priorität. Günstige Checks zuerst, teure nur bei Bedarf.

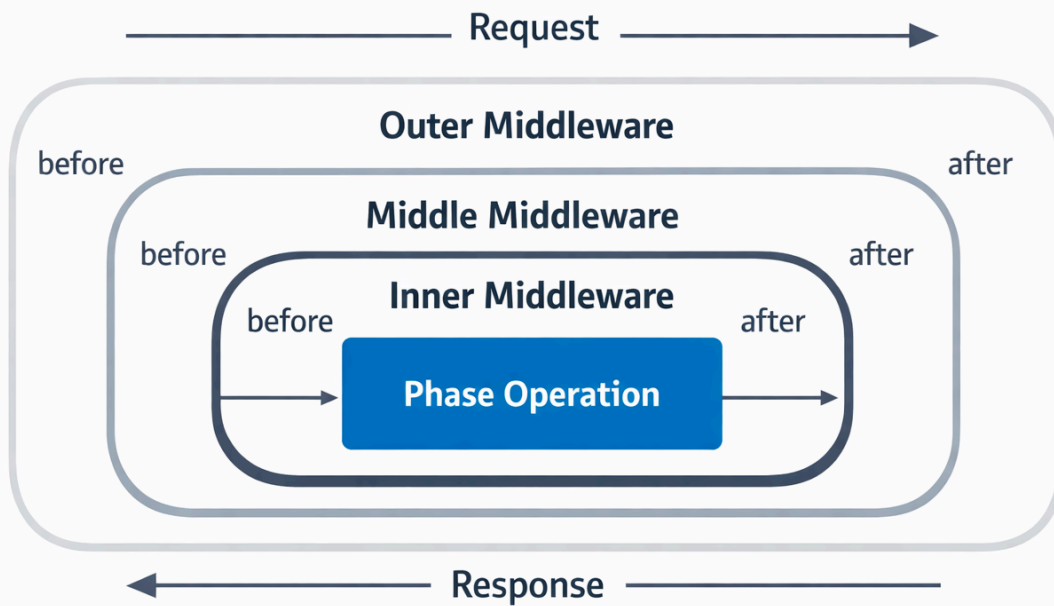
Wann welche Strategie?

- **Sequential:** Debugging, deterministische Tests, wenn Reviewer-Reihenfolge wichtig ist.
- **Parallel:** Alle Reviewer sind unabhängig und schnell.
- **FailFast:** Mehrere teure KI-Reviewer, bei denen man bei erstem Fehler abrechnen will.
- **PriorityOrdered:** Mix aus schnellen Syntax-Checks (Priorität 10) und langsamen KI-Reviews (Priorität 100).



Middleware-Pipeline

GEEF unterstützt eine ASP.NET-Core-artige Middleware-Pipeline für Cross-Cutting Concerns:



Jede Middleware sieht einen `GeefMiddlewareContext` mit Phase, RunContext, RunId und CancellationToken. Middleware kann:

- Vor/nach der Phase Code ausführen (Tracing, Logging)
- Die Phase abbrechen (Timeout, Fehlerbehandlung)
- Den CancellationToken ersetzen (z.B. für Phase-spezifische Timeouts)

```
public interface IGeefMiddleware
{
    Task InvokeAsync(GeefMiddlewareContext context, Func<Task> next);
}
```

Mitgelieferte Middleware

Middleware	Zweck
<code>TimeoutMiddleware</code>	Erzwingt Phase-spezifische Timeouts über CancellationToken-Propagation
<code>TracingMiddleware</code>	OpenTelemetry-Spans für jede Phase
<code>ExceptionHandlingMiddleware</code>	Wandelt unerwartete Exceptions in strukturierte Fehler um

Nächste Schritte

- API-Referenz — Detaillierte Referenz aller Typen
- Konfiguration — Policies und Middleware konfigurieren
- Fachartikel — Theoretische Grundlagen und Motivation